DESIGN AND ANALYSIS
OF ALGORITHMS

ASSIGNMENT 3 – CO3

A Comprehensive Study of Divide-and-Conquer Algorithms

Submission detail	**Information**
Student A – Name / Register No.	S.Harshavardhan/192524106
Student B – Name / Register No.	D,Madhusudhan/192524214
Programme / Branch	B.Tech – AIDS
Course / Course Code	Design and Analysis of Algorithms /

DECLARATION AND ACKNOWLEDGEMENT

We declare that this report has been prepared for the Design and Analysis of Algorithms course. Both partners accept responsibility for understanding the complete assignment, including sections primarily prepared by the other partner.

All experiment fields marked Measure or Count are templates. They must be replaced with genuine measurements from the implementation before submission. No timing result in this draft should be represented as an observed result.

We thank our faculty for designing an assignment that connects algorithm design, mathematical analysis, implementation and experimental evaluation.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

ABSTRACT

Divide and Conquer solves a problem by dividing it into smaller subproblems, solving those subproblems recursively and combining their results. This report investigates maximum and minimum, Binary Search, Merge Sort, Quick Sort, Karatsuba multiplication, Strassen matrix multiplication, Closest Pair, Median of Medians and Meet-in-the-Middle.

The report explains decomposition, base cases, returned information, combine operations, pseudocode, correctness reasoning and asymptotic complexity. It also provides reproducible experimental plans and tables.

The main conclusion is that better asymptotic complexity does not guarantee faster execution for every input. Recursion, memory, pivot quality, padding, cache behaviour and constant factors affect practical performance.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

CONTENTS AND REPORT ROADMAP

This report is organised around the seven questions in Assignment 3 – CO3. Q1 studies extrema and binary search. Q2 compares two sorting algorithms. Q3 and Q4 examine faster arithmetic algorithms. Q5 studies a geometric problem. Q6 investigates selection and subset sum. Q7 demonstrates a first-principles design process.

Each experimental table is intentionally provided as a fill-in record. Students should attach code, console output, plots and screenshots generated on their own machine.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

INTRODUCTION TO ALGORITHM ANALYSIS

An algorithm is a finite sequence of unambiguous steps that transforms input into output. Algorithm analysis estimates the resources required as input size grows. Time complexity describes computational work, while space complexity describes additional memory.

Big-O gives an asymptotic upper bound; Big-Theta gives a tight asymptotic bound; Big-Omega gives a lower bound. These notations describe growth rather than exact elapsed seconds.

Divide and Conquer has three stages: divide the instance, conquer the smaller instances, and combine their results. A base case prevents infinite recursion. The strategy is effective when subproblems are smaller and the combine operation is efficient.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

METHODOLOGY AND EXPERIMENTAL CONTROLS

Use the same programming language, machine, input data and measurement method for competing algorithms. Record the processor, memory, operating system, language version and compiler or interpreter settings.

Test multiple input sizes and distributions. Run each timing case repeatedly and report the median or average consistently. Count operations separately from elapsed time because operation counts are less affected by system load.

Use fixed random seeds for reproducibility. Verify outputs against a trusted baseline. Never describe predicted values as experimental measurements.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q1 – MAXIMUM AND MINIMUM PROBLEM

Given an array of n values, find both its smallest and largest elements. A direct scan can update minimum and maximum in one pass. The D&C strategy divides the array into two segments, recursively obtains the extrema of each segment and combines the results.

Each recursive call returns a pair (minimum, maximum). For one element, both returned values equal that element. For two elements, one comparison determines both extrema. For larger segments, compare the two local minima and the two local maxima.

Correctness follows by induction: if both halves return their true extrema, the smaller of the two minima is the global minimum and the larger of the two maxima is the global maximum.

The important design decision is what information a recursive call must return. Returning only the maximum would not be sufficient to compute the global minimum without additional work. Returning both extrema makes each subproblem self-contained and allows the parent call to combine two compact summaries.

The tournament interpretation gives an intuitive view: each pair of values competes to determine a local winner and loser. Winners continue in the maximum tournament and losers continue in the minimum tournament. The pairwise approach avoids comparing every element separately against both current extrema.

During implementation, test negative values, repeated values, already ordered arrays, odd-sized arrays and arrays containing one element. The returned minimum and maximum should match Python's min and max or an equivalent trusted baseline. The comparison counter should be incremented only when two data values are actually compared.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q1 – MAXIMUM AND MINIMUM PSEUDOCODE

MAXMIN(A,l,h): if l=h return (A[l],A[l]). If h=l+1, compare the two elements and return (smaller,larger). Otherwise compute m=(l+h)//2; recursively obtain (min1,max1) and (min2,max2); return (min(min1,min2), max(max1,max2)).

For n a power of two using a two-element base case, the comparison recurrence is $C(n)=2C(n/2)+2$ with $C(2)=1$. Its solution is $C(n)=3n/2-2$. Two independent scans typically require $2n-2$ comparisons.

The D&C algorithm is $\Theta(n)$ time and uses $\Theta(\log n)$ recursion stack for balanced splitting. Exact comparison counts for arbitrary odd n depend on the base-case and pairing strategy.

The important design decision is what information a recursive call must return. Returning only the maximum would not be sufficient to compute the global minimum without additional work. Returning both extrema makes each subproblem self-contained and allows the parent call to combine two compact summaries.

The tournament interpretation gives an intuitive view: each pair of values competes to determine a local winner and loser. Winners continue in the maximum tournament and losers continue in the minimum tournament. The pairwise approach avoids comparing every element separately against both current extrema.

During implementation, test negative values, repeated values, already ordered arrays, odd-sized arrays and arrays containing one element. The returned minimum and maximum should match Python's min and max or an equivalent trusted baseline. The comparison counter should be incremented only when two data values are actually compared.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q1 – COMPARISON COUNT TABLE

The following table contains theoretical comparison counts for the stated even-size recurrence. It is not measured timing data. For non-power-of-two inputs, record the actual count from the implementation.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

n	**D&C comparisons**	**Two independent scans**
2	1	2
4	4	6
8	10	14
16	22	30
32	46	62
64	94	126

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q1 – BINARY SEARCH

Binary Search locates a target in a sorted array. Compare the target with the middle element. If equal, return its index. If smaller, continue in the left half; if larger, continue in the right half.

The sorted order proves that the discarded half cannot contain the target. The search terminates successfully when a match is found or unsuccessfully when low exceeds high.

The recurrence is $T(n)=T(n/2)+\Theta(1)$, so the time complexity is $\Theta(\log n)$. The iterative implementation uses $\Theta(1)$ auxiliary space. The sorted-input condition is essential.

The key invariant is that, whenever the loop begins, if the target exists in the array then at least one occurrence lies between the current low and high indices. Initially this interval contains the complete array, so the invariant is true. Each comparison either finds the target or removes a region that cannot contain it because the array is sorted. Therefore the invariant is preserved until the interval becomes empty or a match is found.

The midpoint should be calculated as low + (high − low) // 2 rather than (low + high) // 2 in languages where integer overflow is possible. The former expression avoids adding two potentially large indices. Boundary updates must use mid−1 and mid+1; failing to move beyond the midpoint can cause an infinite loop.

For an array of size n, after k unsuccessful comparisons the remaining interval is at most approximately n/2^k. The search stops once this quantity is below one, so k is proportional to $\log_2 n$. This explains why doubling the input size adds only about one comparison to the worst-case search path.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q1 – BINARY SEARCH PSEUDOCODE AND TESTS

Set low=0 and high=n−1. While low≤high, compute mid=low+(high−low)//2. If A[mid]=x return mid. If x<A[mid], set high=mid−1; otherwise set low=mid+1. If the loop ends, return NOT_FOUND.

Test first, middle and last elements; missing values between existing values; values below and above the range; duplicate values; and an empty array. Decide whether the implementation returns any matching duplicate or the first occurrence.

Record the number of comparisons for successful and unsuccessful searches as n increases.

The key invariant is that, whenever the loop begins, if the target exists in the array then at least one occurrence lies between the current low and high indices. Initially this interval contains the complete array, so the invariant is true. Each comparison either finds the target or removes a region that cannot contain it because the array is sorted. Therefore the invariant is preserved until the interval becomes empty or a match is found.

The midpoint should be calculated as low + (high − low) // 2 rather than (low + high) // 2 in languages where integer overflow is possible. The former expression avoids adding two potentially large indices. Boundary updates must use mid−1 and mid+1; failing to move beyond the midpoint can cause an infinite loop.

For an array of size n, after k unsuccessful comparisons the remaining interval is at most approximately n/2^k. The search stops once this quantity is below one, so k is proportional to $\log_2 n$. This explains why doubling the input size adds only about one comparison to the worst-case search path.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q2 – MERGE SORT

Merge Sort repeatedly divides the array into two approximately equal halves until each segment has zero or one element. It then merges two sorted segments by repeatedly selecting the smaller front element.

The recurrence is $T(n)=2T(n/2)+\Theta(n)$, because two half-size recursive calls are followed by a linear merge. The Master Theorem gives $\Theta(n \log n)$.

Standard array Merge Sort requires $\Theta(n)$ auxiliary memory and can be stable. Its balanced decomposition makes its worst-case running time predictable.

A recursion tree helps explain the difference between the two sorting methods. Merge Sort has approximately $\log_2 n$ levels because each split is balanced. At every level, the total number of elements processed by merging is proportional to n. Multiplying the work per level by the number of levels gives $n \log n$.

Quick Sort's recursion tree depends on the pivot rank. A pivot near the median produces two similarly sized branches and a shallow tree. A pivot that is repeatedly the smallest or largest element produces one branch containing almost all remaining elements. The sum $n+(n-1)+(n-2)+\ldots+1$ is proportional to n^2.

The experiment should record partition sizes, not merely total execution time. Recursion depth provides direct evidence of decomposition quality. If the depth approaches n for an input, the partition sequence is highly unbalanced. A randomized pivot or median-of-three strategy can reduce the likelihood of this behaviour.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q2 – QUICK SORT

Quick Sort chooses a pivot, partitions the array around it and recursively sorts the resulting partitions. For this assignment, state the pivot strategy explicitly. A suitable baseline is Lomuto partitioning with the last element as pivot.

Balanced partitions lead to $T(n)=2T(n/2)+\Theta(n)=\Theta(n \log n)$. Repeated partitions of sizes zero and $n-1$ lead to $T(n)=T(n-1)+\Theta(n)=\Theta(n^2)$.

Sorted or reverse-sorted inputs can be problematic for a deterministic endpoint pivot. Randomized pivot selection reduces dependence on input order but does not create a deterministic worst-case guarantee.

A recursion tree helps explain the difference between the two sorting methods. Merge Sort has approximately $\log_2 n$ levels because each split is balanced. At every level, the total number of elements processed by merging is proportional to n. Multiplying the work per level by the number of levels gives $n \log n$.

Quick Sort's recursion tree depends on the pivot rank. A pivot near the median produces two similarly sized branches and a shallow tree. A pivot that is repeatedly the smallest or largest element produces one branch containing almost all remaining elements. The sum $n+(n-1)+(n-2)+\ldots+1$ is proportional to n^2.

The experiment should record partition sizes, not merely total execution time. Recursion depth provides direct evidence of decomposition quality. If the depth approaches n for an input, the partition sequence is highly unbalanced. A randomized pivot or median-of-three strategy can reduce the likelihood of this behaviour.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q2 – SORTING COMPARISON

Merge Sort has a predictable $\Theta(n \log n)$ worst-case bound and generally requires additional array storage. Quick Sort is often in-place apart from recursion stack, but its performance depends on partition quality and pivot selection.

Experiment with random, sorted, reverse-sorted and duplicate-heavy arrays. Record comparisons, swaps, partition sizes, maximum recursion depth and median elapsed time.

For applications with unknown input characteristics and a strict worst-case requirement, Merge Sort is a predictable choice. Quick Sort can be attractive when memory locality and average practical performance matter, with appropriate safeguards.

A recursion tree helps explain the difference between the two sorting methods. Merge Sort has approximately $\log_2 n$ levels because each split is balanced. At every level, the total number of elements processed by merging is proportional to n. Multiplying the work per level by the number of levels gives n log n.

Quick Sort's recursion tree depends on the pivot rank. A pivot near the median produces two similarly sized branches and a shallow tree. A pivot that is repeatedly the smallest or largest element produces one branch containing almost all remaining elements. The sum $n+(n-1)+(n-2)+\ldots+1$ is proportional to n^2.

The experiment should record partition sizes, not merely total execution time. Recursion depth provides direct evidence of decomposition quality. If the depth approaches n for an input, the partition sequence is highly unbalanced. A randomized pivot or median-of-three strategy can reduce the likelihood of this behaviour.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q2 – SORTING EXPERIMENT RECORD

Use equivalent copies of each input for both algorithms. Do not let one algorithm receive an already sorted array unless that is the deliberate test case. Include the exact pivot strategy in the report.

A recursion tree helps explain the difference between the two sorting methods. Merge Sort has approximately $\log_2 n$ levels because each split is balanced. At every level, the total number of elements processed by merging is proportional to n. Multiplying the work per level by the number of levels gives n log n.

Quick Sort's recursion tree depends on the pivot rank. A pivot near the median produces two similarly sized branches and a shallow tree. A pivot that is repeatedly the smallest or largest element produces one branch containing almost all remaining elements. The sum $n+(n-1)+(n-2)+\ldots+1$ is proportional to n^2.

The experiment should record partition sizes, not merely total execution time. Recursion depth provides direct evidence of decomposition quality. If the depth approaches n for an input, the partition sequence is highly unbalanced. A randomized pivot or median-of-three strategy can reduce the likelihood of this behaviour.

Input size	Distribution	Algorithm	Comparisons	Max depth	Median time
1,000	Random	Merge / Quick	Measure	Measure	Measure
1,000	Sorted	Merge / Quick	Measure	Measure	Measure
10,000	Reverse	Merge / Quick	Measure	Measure	Measure
50,000	Duplicates	Merge / Quick	Measure	Measure	Measure

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q3 – KARATSUBA MULTIPLICATION

Grade-school multiplication of two n-digit numbers uses $\Theta(n^2)$ single-digit products. Karatsuba reduces the number of recursive multiplications by splitting each number into high and low parts.

Write X=aB^m+b and Y=cB^m+d. Direct expansion requires ac, ad, bc and bd. Karatsuba computes z2=ac, z0=bd and z1=(a+b)(c+d). The cross term is z1−z2−z0.

The identity is valid because (a+b)(c+d)=ac+ad+bc+bd. Subtracting ac and bd leaves ad+bc.

The decomposition uses positional notation. Splitting at m digits means the high part is multiplied by the base raised to m, while the low part contains the final m digits. For decimal integers the base is ten; for binary representations the base is two. The algebra is independent of the chosen base.

The three recursive products are smaller than the original product, although the sums a+b and c+d can occasionally contain one extra digit. A careful implementation must handle this additional digit and use a consistent split position when operand lengths differ.

A reliable benchmark should separate correctness from speed. First compare the numerical answer with ordinary multiplication for many generated operands. Then benchmark only verified inputs. Use sufficiently large operands, repeat the test, and state the threshold at which the implementation switches to the schoolbook method.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q3 – KARATSUBA RECURRENCE

The result is $z2 \cdot B^{(2m)} + (z1 - z2 - z0) \cdot B^{m} + z0$. The recurrence is $T(n) = 3T(n/2) + \Theta(n)$, since three recursive multiplications are combined using linear-time additions, subtractions and shifts.

The Master Theorem gives $\Theta(n^{(\log_2 3)})$, approximately $\Theta(n^{1.585})$, compared with $\Theta(n^2)$ for standard multiplication.

Karatsuba can be slower for small operands because recursion, splitting, allocation and additions have overhead. Practical implementations switch to ordinary multiplication below a threshold.

The decomposition uses positional notation. Splitting at m digits means the high part is multiplied by the base raised to m, while the low part contains the final m digits. For decimal integers the base is ten; for binary representations the base is two. The algebra is independent of the chosen base.

The three recursive products are smaller than the original product, although the sums a+b and c+d can occasionally contain one extra digit. A careful implementation must handle this additional digit and use a consistent split position when operand lengths differ.

A reliable benchmark should separate correctness from speed. First compare the numerical answer with ordinary multiplication for many generated operands. Then benchmark only verified inputs. Use sufficiently large operands, repeat the test, and state the threshold at which the implementation switches to the schoolbook method.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q3 – KARATSUBA EXPERIMENT

Test operands with increasing digit lengths such as 16, 32, 64, 128, 256, 512 and 1,024 digits, subject to language limits. Generate operands of equal length and verify every product against standard multiplication.

Repeat timing runs and record multiplication counts, recursive calls and the threshold used. If no crossover appears, increase the sizes or improve the implementation, but do not invent a crossover point.

The decomposition uses positional notation. Splitting at m digits means the high part is multiplied by the base raised to m, while the low part contains the final m digits. For decimal integers the base is ten; for binary representations the base is two. The algebra is independent of the chosen base.

The three recursive products are smaller than the original product, although the sums a+b and c+d can occasionally contain one extra digit. A careful implementation must handle this additional digit and use a consistent split position when operand lengths differ.

A reliable benchmark should separate correctness from speed. First compare the numerical answer with ordinary multiplication for many generated operands. Then benchmark only verified inputs. Use sufficiently large operands, repeat the test, and state the threshold at which the implementation switches to the schoolbook method.

Digits	**Standard time**	**Karatsuba time**	**Standard products**	**Karatsuba calls**
16	Measure	Measure	Count	Count
32	Measure	Measure	Count	Count
64	Measure	Measure	Count	Count
128	Measure	Measure	Count	Count
256	Measure	Measure	Count	Count
512	Measure	Measure	Count	Count

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or

machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q4 – STRASSEN MATRIX MULTIPLICATION

Conventional multiplication of two n×n matrices requires $\Theta(n^3)$ scalar multiplications. Strassen divides each matrix into four blocks and computes seven block products instead of eight.

Define M1=(A11+A22)(B11+B22), M2=(A21+A22)B11, M3=A11(B12−B22), M4=A22(B21−B11), M5=(A11+A12)B22, M6=(A21−A11)(B11+B12), and M7=(A12−A22)(B21+B22).

Then C11=M1+M4−M5+M7; C12=M3+M5; C21=M2+M4; C22=M1−M2+M3+M6. Expanding and cancelling terms verifies the formulas.

Strassen's improvement comes from reducing recursive multiplications, not from eliminating all arithmetic work. The seven block products are combined using additions and subtractions of matrices. Since adding two blocks takes quadratic time in the block dimension, this work contributes $\Theta(n^2)$ at each recursive level.

The seven product identities should be implemented exactly as defined and checked on small matrices where the result can be calculated manually. A sign error in any expression can produce incorrect output even though the recursion and dimensions appear valid.

Padding a matrix to the next power-of-two dimension simplifies recursive splitting but may create a much larger temporary matrix. For example, a dimension just above a power of two may require padding nearly to twice that dimension. This overhead is one reason a practical implementation uses a threshold and conventional multiplication for small blocks.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q4 – STRASSEN COMPLEXITY AND ENGINEERING

The recurrence is $T(n)=7T(n/2)+\Theta(n^2)$, giving $\Theta(n^{\log_2 7})$ or approximately $\Theta(n^{2.807})$. The $\Theta(n^2)$ term represents block additions and subtractions.

Dimensions that are not powers of two can be padded with zero rows and columns, then the output is trimmed. Padding can increase work and memory. Recursion and temporary arrays also create overhead.

Conventional multiplication can remain faster for small matrices or when a highly optimized library is available. Measure the crossover on the target hardware.

Strassen's improvement comes from reducing recursive multiplications, not from eliminating all arithmetic work. The seven block products are combined using additions and subtractions of matrices. Since adding two blocks takes quadratic time in the block dimension, this work contributes $\Theta(n^2)$ at each recursive level.

The seven product identities should be implemented exactly as defined and checked on small matrices where the result can be calculated manually. A sign error in any expression can produce incorrect output even though the recursion and dimensions appear valid.

Padding a matrix to the next power-of-two dimension simplifies recursive splitting but may create a much larger temporary matrix. For example, a dimension just above a power of two may require padding nearly to twice that dimension. This overhead is one reason a practical implementation uses a threshold and conventional multiplication for small blocks.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q4 – STRASSEN VALIDATION AND RESULTS

Test 1×1, 2×2, 3×3, 4×4, 5×5 and 7×7 matrices. Include zero matrices, identity matrices, negative values and random matrices. Compare every result element with conventional multiplication.

Record whether padding is needed, multiplication counts, memory use and repeated execution times. Use the same numeric type and matrix values for both methods.

Strassen's improvement comes from reducing recursive multiplications, not from eliminating all arithmetic work. The seven block products are combined using additions and subtractions of matrices. Since adding two blocks takes quadratic time in the block dimension, this work contributes $\Theta(n^2)$ at each recursive level.

The seven product identities should be implemented exactly as defined and checked on small matrices where the result can be calculated manually. A sign error in any expression can produce incorrect output even though the recursion and dimensions appear valid.

Padding a matrix to the next power-of-two dimension simplifies recursive splitting but may create a much larger temporary matrix. For example, a dimension just above a power of two may require padding nearly to twice that dimension. This overhead is one reason a practical implementation uses a threshold and conventional multiplication for small blocks.

Size	**Conventional time**	**Strassen time**	**Padding**	**Correct?**
16×16	Measure	Measure	No	Yes/No
32×32	Measure	Measure	No	Yes/No
64×64	Measure	Measure	No	Yes/No
128×128	Measure	Measure	No	Yes/No
127×127	Measure	Measure	128	Yes/No

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q5 – CLOSEST PAIR OF POINTS

Given n points in a plane, find the pair with the minimum Euclidean distance. Brute force checks $n(n-1)/2$ pairs, requiring $\Theta(n^2)$ time.

The D&C algorithm sorts points by x-coordinate, divides them into left and right halves, recursively finds the closest pair in each half, and checks a narrow strip around the dividing line.

Let δ be the smaller recursive distance. A cross-boundary pair closer than δ must lie within δ horizontally of the dividing line. The strip is processed in y-order; geometric packing limits the number of relevant neighbours per point to a constant in the standard algorithm.

The main challenge is maintaining the points sorted by y-coordinate. If the algorithm sorts the strip independently at every recursive call, the combine stage may cost $\Theta(n \log n)$ rather than $\Theta(n)$, weakening the overall recurrence. A standard implementation creates x-sorted and y-sorted lists once and partitions the y-list consistently at each level.

The strip argument relies on geometry. Since each half already has no pair closer than δ, a small rectangle around a point can contain only a limited number of candidate points without contradicting that minimum-distance property. This packing argument makes a constant-neighbour comparison sufficient.

Special care is required when points have equal x-coordinates or identical coordinates. Use a consistent tie-breaking rule, such as x followed by y and then an identifier, so that each point is assigned to exactly one recursive half. Identical points should immediately permit a distance of zero.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q5 – CLOSEST PAIR CORRECTNESS

Every closest pair lies wholly in the left half, wholly in the right half, or crosses the dividing boundary. The recursive calls handle the first two cases. Any closer crossing pair must lie in the strip and is considered by the bounded neighbour scan.

With initial sorting and y-order maintained through recursion, the recurrence is $T(n)=2T(n/2)+\Theta(n)$, giving $\Theta(n \log n)$. Re-sorting at every level can add avoidable overhead.

Use brute force for small base cases. Test duplicate coordinates, points on a line, vertical alignment, clustered points and random points.

The main challenge is maintaining the points sorted by y-coordinate. If the algorithm sorts the strip independently at every recursive call, the combine stage may cost $\Theta(n \log n)$ rather than $\Theta(n)$, weakening the overall recurrence. A standard implementation creates x-sorted and y-sorted lists once and partitions the y-list consistently at each level.

The strip argument relies on geometry. Since each half already has no pair closer than δ, a small rectangle around a point can contain only a limited number of candidate points without contradicting that minimum-distance property. This packing argument makes a constant-neighbour comparison sufficient.

Special care is required when points have equal x-coordinates or identical coordinates. Use a consistent tie-breaking rule, such as x followed by y and then an identifier, so that each point is assigned to exactly one recursive half. Identical points should immediately permit a distance of zero.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q5 – CLOSEST PAIR EXPERIMENT

Compare both algorithms on identical point sets. Record distance calculations and elapsed time. The evidence that D&C reduces work is the count of distance calculations and the growth trend as n increases.

Measure the practical crossover instead of assuming one. Include correctness checks by comparing the distance returned by D&C with the brute-force answer for manageable n.

The main challenge is maintaining the points sorted by y-coordinate. If the algorithm sorts the strip independently at every recursive call, the combine stage may cost $\Theta(n \log n)$ rather than $\Theta(n)$, weakening the overall recurrence. A standard implementation creates x-sorted and y-sorted lists once and partitions the y-list consistently at each level.

The strip argument relies on geometry. Since each half already has no pair closer than δ, a small rectangle around a point can contain only a limited number of candidate points without contradicting that minimum-distance property. This packing argument makes a constant-neighbour comparison sufficient.

Special care is required when points have equal x-coordinates or identical coordinates. Use a consistent tie-breaking rule, such as x followed by y and then an identifier, so that each point is assigned to exactly one recursive half. Identical points should immediately permit a distance of zero.

n	Brute checks	D&C checks	Brute time	D&C time
100	Count	Count	Measure	Measure
500	Count	Count	Measure	Measure
1000	Count	Count	Measure	Measure
5000	Count	Count	Measure	Measure
10000	Count	Count	Measure	Measure

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q6 – MEDIAN OF MEDIANS

Selection finds the k-th smallest element without fully sorting the array. QuickSelect partitions around a pivot and recurses only into the partition containing rank k. A poor pivot can repeatedly leave almost the whole input and cause $\Theta(n^2)$ worst-case time.

Median of Medians divides the elements into groups of five, finds each group median, recursively selects the median of those medians, and partitions around that pivot.

The pivot guarantees a constant fraction of elements can be discarded. A standard recurrence is bounded by $T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + \Theta(n)$, which solves to $\Theta(n)$ worst-case time.

Grouping into fives is useful because the median of each group provides a small certificate about how many elements are no smaller and no larger than that median. The median of these medians therefore cannot be arbitrarily close to an extreme of the complete array.

After partitioning, the algorithm must update k correctly when it recurses into the greater-than-pivot region. If the less-than region contains L elements and the equal region contains E elements, the rank within the greater region becomes $k - L - E$ when using a one-based rank convention.

Duplicate values should be handled as a separate equal-to-pivot region. This three-way partition prevents repeated recursion over values equal to the pivot and makes the implementation more robust on duplicate-heavy inputs.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q6 – SELECTION COMPARISON

Randomized QuickSelect has expected $\Theta(n)$ time but $\Theta(n^2)$ worst-case time. Median of Medians has deterministic $\Theta(n)$ worst-case time but additional pivot-construction overhead.

Test random, sorted, reverse-sorted and duplicate-heavy arrays. Test k at the first, middle and last ranks. Compare each answer with the corresponding item in a fully sorted copy.

Pivot quality controls the amount of input discarded after partitioning. A good pivot makes progress; a poor pivot may leave a large recursive subproblem.

Grouping into fives is useful because the median of each group provides a small certificate about how many elements are no smaller and no larger than that median. The median of these medians therefore cannot be arbitrarily close to an extreme of the complete array.

After partitioning, the algorithm must update k correctly when it recurses into the greater-than-pivot region. If the less-than region contains L elements and the equal region contains E elements, the rank within the greater region becomes $k-L-E$ when using a one-based rank convention.

Duplicate values should be handled as a separate equal-to-pivot region. This three-way partition prevents repeated recursion over values equal to the pivot and makes the implementation more robust on duplicate-heavy inputs.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q6 – MEDIAN OF MEDIANS DATA RECORD

Instrument comparisons and recursion sizes. Explain the group-of-five strategy, how the pivot is selected, and why the algorithm avoids consistently disastrous partitions.

Grouping into fives is useful because the median of each group provides a small certificate about how many elements are no smaller and no larger than that median. The median of these medians therefore cannot be arbitrarily close to an extreme of the complete array.

After partitioning, the algorithm must update k correctly when it recurses into the greater-than-pivot region. If the less-than region contains L elements and the equal region contains E elements, the rank within the greater region becomes k−L−E when using a one-based rank convention.

Duplicate values should be handled as a separate equal-to-pivot region. This three-way partition prevents repeated recursion over values equal to the pivot and makes the implementation more robust on duplicate-heavy inputs.

n	**k**	**QuickSelect comparisons**	**Median-of-Medians comparisons**	**Correct?**
100	50	Measure	Measure	Yes/No
1000	500	Measure	Measure	Yes/No
10000	5000	Measure	Measure	Yes/No
50000	25000	Measure	Measure	Yes/No

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q6 – MEET-IN-THE-MIDDLE SUBSET SUM

Subset Sum asks whether a subset of n numbers totals a target S. Straight enumeration checks up to 2^n subsets. Meet-in-the-Middle splits the values into two halves and generates all subset sums for each half.

Store the sums from one half in a searchable structure. For each sum x from the other half, search for target−x. The method reduces the exponential search from approximately 2^n to approximately 2^(n/2) times polynomial factors.

The trade-off is memory: each half produces about 2^(n/2) subset sums. This is useful when time is the main limitation and sufficient memory is available.

Each subset sum can be generated efficiently by starting with the sum zero and extending existing sums with the next value. The empty subset must be included because it may be needed when the target is formed entirely by the other half.

A sorted list supports binary search for the complementary value target−x. A hash set can also provide expected constant-time membership checks, although its memory overhead and worst-case guarantees differ from sorting and binary search.

The method is not polynomial-time: its running time remains exponential, but the exponent is approximately halved. The technique is useful for moderate n when 2^n is too large but 2^(n/2) values can still be generated and stored.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q6 – MEET-IN-THE-MIDDLE ANALYSIS

For n=40, each half has 2^20, or about 1,048,576, subsets. This is much smaller than 2^40 complete subsets, but storing more than a million sums still requires significant memory.

Sort one half's sums and binary-search for the complement, or use hashing. Sorting costs $O(2^{(n/2)} \cdot n)$ in a straightforward estimate; memory is $\Theta(2^{(n/2)})$.

Test negative values, zero values, repeated values, empty subsets and targets that are both achievable and impossible.

Each subset sum can be generated efficiently by starting with the sum zero and extending existing sums with the next value. The empty subset must be included because it may be needed when the target is formed entirely by the other half.

A sorted list supports binary search for the complementary value target−x. A hash set can also provide expected constant-time membership checks, although its memory overhead and worst-case guarantees differ from sorting and binary search.

The method is not polynomial-time: its running time remains exponential, but the exponent is approximately halved. The technique is useful for moderate n when 2^n is too large but 2^(n/2) values can still be generated and stored.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q7 – FIRST-PRINCIPLES DESIGN CHALLENGE

The supplied assignment brief describes the stages for Q7 but does not name a specific problem. This report therefore uses Maximum Subarray Sum as an explicitly selected case study. Confirm with faculty that this is the intended or acceptable problem if a separate in-class problem was given.

Problem: given an integer array, find a non-empty contiguous subarray with maximum sum. For [−2,1,−3,4,−1,2,1,−5,4], the maximum subarray is [4,−1,2,1] and its sum is 6.

Student A initial proposal: split at the midpoint; base case one element; return best sum and indices; combine left, right and crossing candidates. Student B initial proposal: split at midpoint; find best left suffix and right prefix; combine their sum with the two recursive answers. Replace these templates with the pair's actual independent proposals.

The design process begins by specifying exactly what one recursive call receives and what it returns. This interface is essential because the parent call must have enough information to construct a correct answer. A vague statement such as 'solve both halves' is incomplete unless the returned values and combine procedure are identified.

The correctness proof should follow the same structure as the algorithm. Prove the base case directly. Then assume the recursive calls are correct and show that the combine operation considers every possible location of an optimal solution. This establishes correctness for the full input by induction.

The experimental baseline should be simple and independently implemented. Comparing the D&C algorithm against a trusted brute-force method on small inputs is particularly valuable because it checks correctness as well as performance. Larger inputs can then be used to study growth when the baseline remains feasible.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q7 – JOINT DESIGN AND RECURRENCE

The joint solution computes the best subarray wholly in the left half, wholly in the right half, and crossing the midpoint. The crossing candidate is the best suffix of the left half plus the best prefix of the right half.

The base case is one element, returned with its sum and indices. The combine stage scans from the midpoint leftwards and from midpoint+1 rightwards, retaining the best running sums.

The recurrence is $T(n)=2T(n/2)+\Theta(n)$, because two recursive calls operate on half-size inputs and the crossing scan is linear. Thus the algorithm runs in $\Theta(n \log n)$ time and uses $\Theta(\log n)$ recursion stack.

The design process begins by specifying exactly what one recursive call receives and what it returns. This interface is essential because the parent call must have enough information to construct a correct answer. A vague statement such as 'solve both halves' is incomplete unless the returned values and combine procedure are identified.

The correctness proof should follow the same structure as the algorithm. Prove the base case directly. Then assume the recursive calls are correct and show that the combine operation considers every possible location of an optimal solution. This establishes correctness for the full input by induction.

The experimental baseline should be simple and independently implemented. Comparing the D&C algorithm against a trusted brute-force method on small inputs is particularly valuable because it checks correctness as well as performance. Larger inputs can then be used to study growth when the baseline remains feasible.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

Q7 – TESTING AND DESIGN EVALUATION

Test a one-element array, all-negative values, all-positive values, an optimum touching the first or last element, repeated equal maxima and mixed positive/negative values. Compare with a simple $O(n^2)$ or $O(n^3)$ baseline on small arrays.

The pair should preserve both independent proposals, record where they agree and differ, explain which ideas were retained, and include the final pseudocode and recurrence.

D&C is appropriate when subproblems are smaller, results can be combined efficiently, and the recursion makes progress. It may be unsuitable when subproblems overlap heavily, combine work is expensive, or memory and recursion overhead dominate.

The design process begins by specifying exactly what one recursive call receives and what it returns. This interface is essential because the parent call must have enough information to construct a correct answer. A vague statement such as 'solve both halves' is incomplete unless the returned values and combine procedure are identified.

The correctness proof should follow the same structure as the algorithm. Prove the base case directly. Then assume the recursive calls are correct and show that the combine operation considers every possible location of an optimal solution. This establishes correctness for the full input by induction.

The experimental baseline should be simple and independently implemented. Comparing the D&C algorithm against a trusted brute-force method on small inputs is particularly valuable because it checks correctness as well as performance. Larger inputs can then be used to study growth when the baseline remains feasible.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

OVERALL COMPLEXITY SUMMARY

The following bounds summarise the algorithms studied. Space estimates are typical and depend on implementation details. These theoretical results should be compared with genuine experimental evidence.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Algorithm	**Time bound**	**Typical auxiliary space**
Max–Min D&C	Θ(n)	Θ(log n) stack
Binary Search	Θ(log n)	Θ(1) iterative
Merge Sort	Θ(n log n)	Θ(n)
Quick Sort	Expected Θ(n log n), worst $\Theta(n^2)$	Recursion stack
Karatsuba	Θ(n^1.585)	Implementation-dependent
Strassen	Θ(n^2.807)	Temporary block storage
Closest Pair	Θ(n log n)	Θ(n) typical
Median of Medians	Θ(n) worst case	Implementation-dependent
Meet-in-the-Middle	About Θ(2^(n/2) poly(n))	Θ(2^(n/2))
Maximum Subarray D&C	Θ(n log n)	Θ(log n) stack

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.

CONCLUSION, CONTRIBUTION AND REFERENCES

Divide and Conquer is effective when decomposition reduces repeated work and the combine stage remains efficient. Binary Search eliminates half the search space. Merge Sort has predictable balanced recursion. Quick Sort demonstrates pivot sensitivity. Karatsuba and Strassen reduce expensive arithmetic operations. Closest Pair uses geometry to avoid checking all pairs. Median of Medians guarantees a useful pivot, while Meet-in-the-Middle trades memory for reduced search time.

The practical performance of an algorithm depends on input size, data distribution, constant factors, memory allocation, recursion, cache behaviour and implementation quality. A complete evaluation therefore combines proof, recurrence analysis, operation counts and repeated timing experiments.

Contribution record: Student A primary – Q1 Max–Min, Q2 Quick Sort, Q3 Karatsuba derivation, Q4 experiments, Q5 D&C design, Q6 Median of Medians, Q7 recurrence. Student B primary – Q1 Binary Search, Q2 Merge Sort analysis, Q3 crossover tests, Q4 Strassen algebra, Q5 comparison evidence, Q6 Meet-in-the-Middle, Q7 implementation. Both students must verify all sections and amend this table to reflect actual work.

Viva preparation: explain a base case; derive a recurrence; describe Quick Sort's worst case; explain Karatsuba's three products; justify Strassen's seven products; explain the closest-pair strip; state Median of Medians' guarantee; explain the Meet-in-the-Middle memory trade-off; and defend the Q7 design.

References: Course handout, Design and Analysis of Algorithms, Assignment 3 – CO3. T. H. Cormen et al., Introduction to Algorithms, 4th ed., MIT Press, 2022. J. Kleinberg and É. Tardos, Algorithm Design, Pearson, 2005. S. Dasgupta, C. Papadimitriou and U. Vazirani, Algorithms, McGraw-Hill, 2008. D. E. Knuth, The Art of Computer Programming, Volume 2, 3rd ed., Addison-Wesley, 1997.

Before submission: add names and register numbers; confirm the Q7 problem with faculty; insert actual source code, screenshots and charts; replace all Measure/Count cells with genuine results; attach the actual independent design proposals; sign the declaration; and verify the final page count.

The analysis should distinguish the mathematical cost of the algorithm from implementation details. The recurrence describes how the amount of work changes as the input is divided. The combine cost must be included because a fast recursive solution can still be inefficient if combining its answers requires repeatedly scanning or sorting large amounts of data.

Correctness should be established independently of complexity. A fast algorithm that returns an incorrect result is not a valid solution. Identify the base case, state the invariant or induction hypothesis, and explain why the combine step preserves the information needed for the final answer.

A complete experimental record should include input size, input characteristics, operation counts, repeated timing results and the machine configuration. Discuss anomalies rather than removing inconvenient results. The final conclusion should be supported by the evidence collected.

Interpretation and submission note: In the final report, connect the explanation above to the implementation completed by the pair. State the input assumptions, describe the chosen data structures, and explain how the output was verified. Where a result depends on a particular input distribution or machine, state that condition clearly. This makes the discussion technically defensible during faculty verification.